Big Data Frameworks

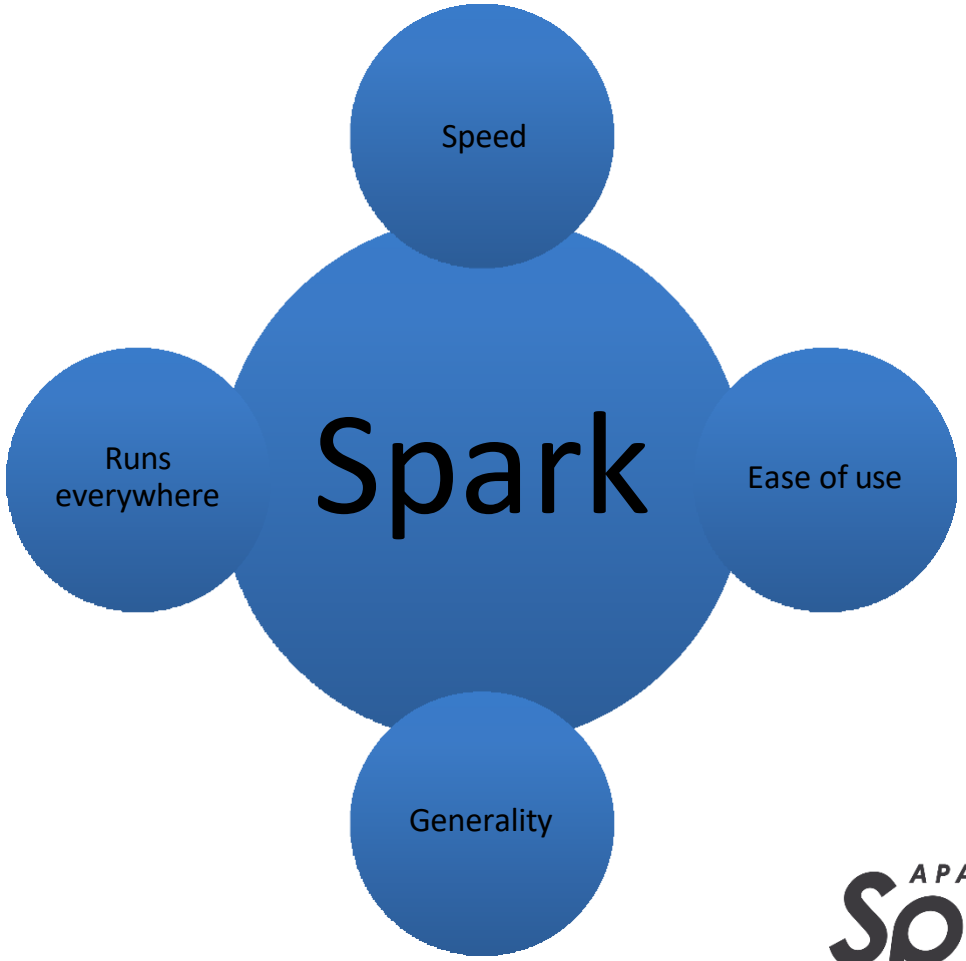
Batch Processing

Marvin Moughabghab

What is Apache Spark ?

Apache Spark is a fast and general engine for large-scale data processing

- Speed: Spark has in-memory processing → very fast (Up to 100x faster than Hadoop)
- Ease of use: Operates easily with Java, Python, Scala, R and SQL
- Generality: Not only does it perform Batch processing, but it can also be used for streaming workload, machine learning and complex analytics
- Runs everywhere: It can run on its own, or with Hadoop, Kubernetes. It can also retrieve data from Cassandra, HDFS, Hive…



What is Apache Spark ?

- Spark is a cluster-computing framework
  - It competes with MapReduce, but can use HDFS to retrieve date
- Spark native language is Scala, but it has official APIs for Java, Python and Spark SQL
- Spark SQL is very similar to SQL
- Spark also has an interactive mode so that developers and users alike can have immediate feedback for queries and other actions
- Relies on a data structure called **Resilient Distributed Dataset**

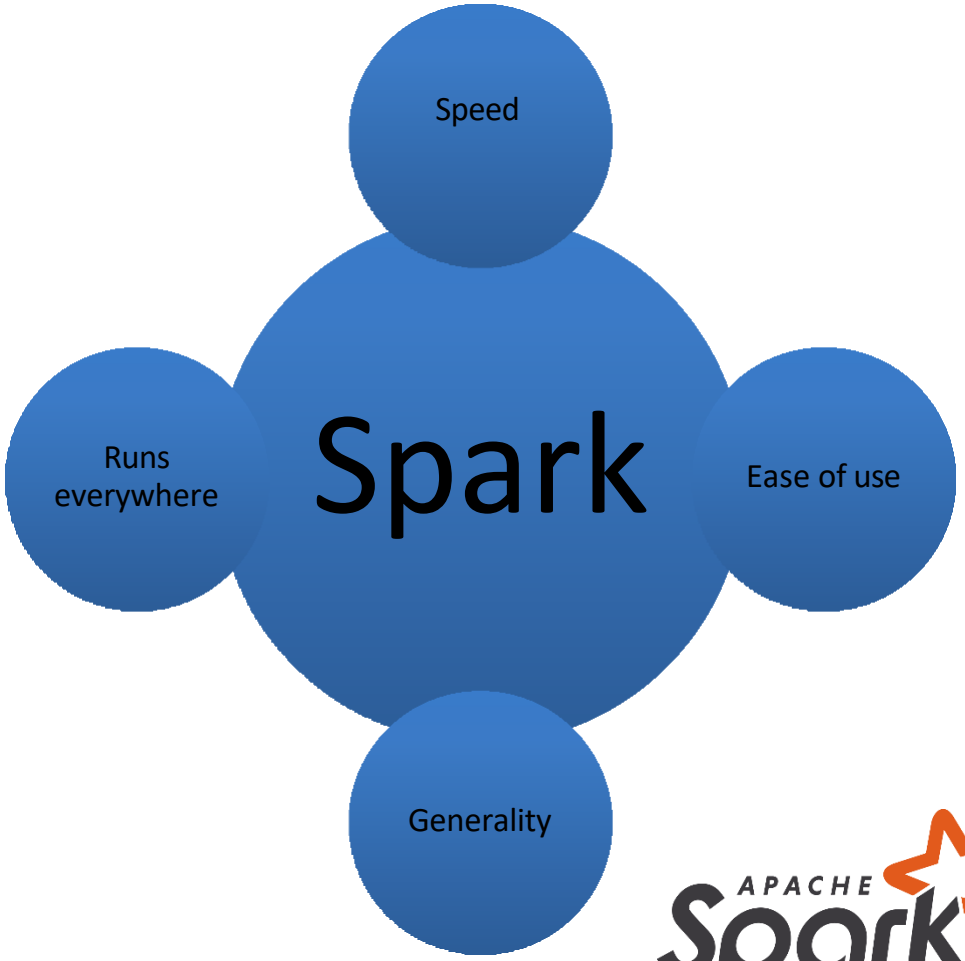
Where Does Spark Fit?

Traditional Approach (Hadoop / MapReduce):

- Data stored in HDFS
- Processing happens **on disk**
- Slower, batch-only processing

With Spark:

- Uses same data sources (HDFS, Hive, etc.)
- Processes data **in memory (RAM)**
- Supports:
  - Batch processing
  - Real-time streaming
  - Machine learning
  - SQL queries



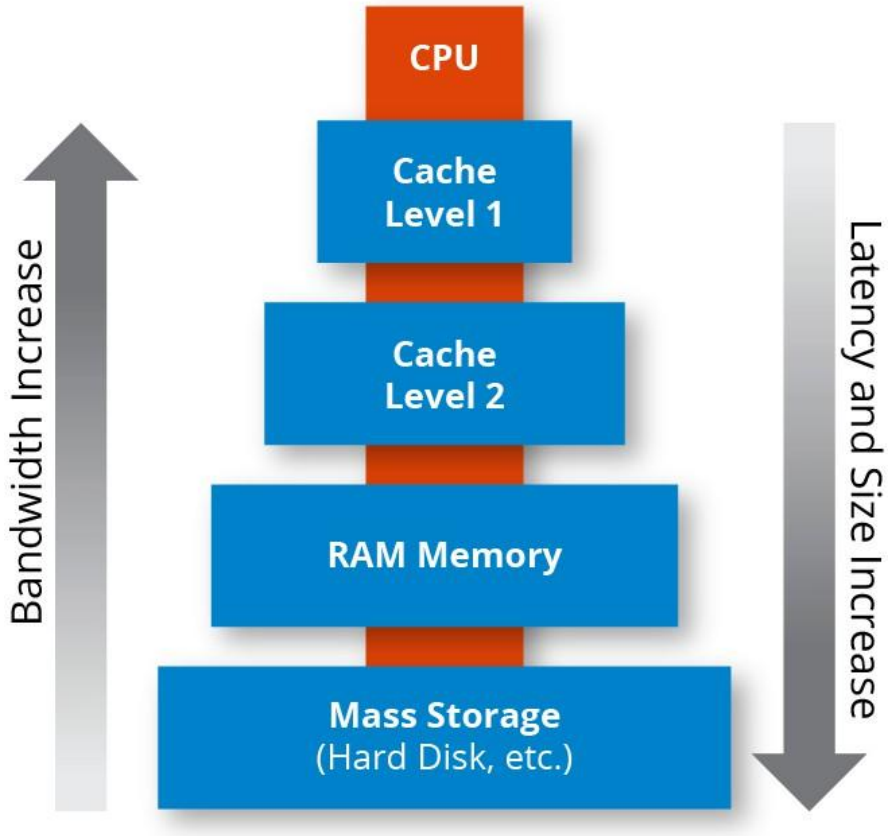
Simple Comparison

Feature	Hadoop (MapReduce)	Spark
Speed	Slow (disk-based)	Fast (in-memory)
Processing	Batch only	Batch + Streaming
Ease of use	Complex	Easier (Python, SQL, etc.)

Think of Spark as an upgraded version of Hadoop processing. Faster, more flexible, and easier to use.

Spark-caching

- Spark allows pulling data sets into a cluster-wide in memory cache
- This comes in handy when data is accessed repeatedly
- >>> variable.cache()
- >>> variable.somefunction()



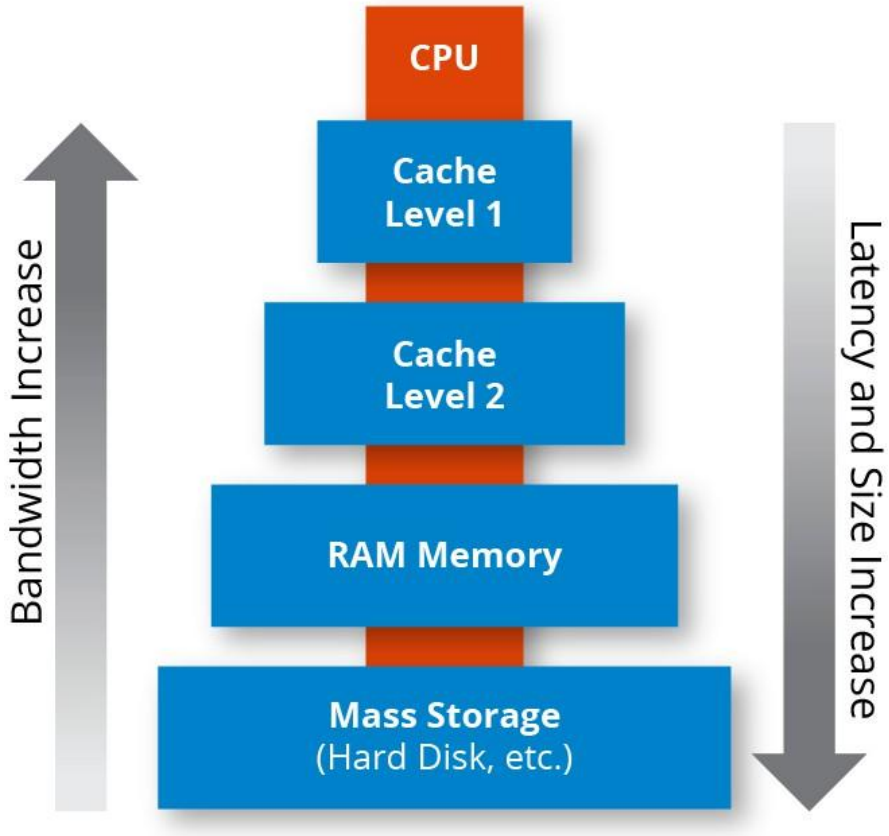
Spark-caching

Without Caching:

- Load dataset
- Perform operation → result
- Run another operation → **data is loaded again (slow)**

With Caching:

- Load dataset once
- Store it in memory (RAM)
- All future operations use the cached data → **much faster**



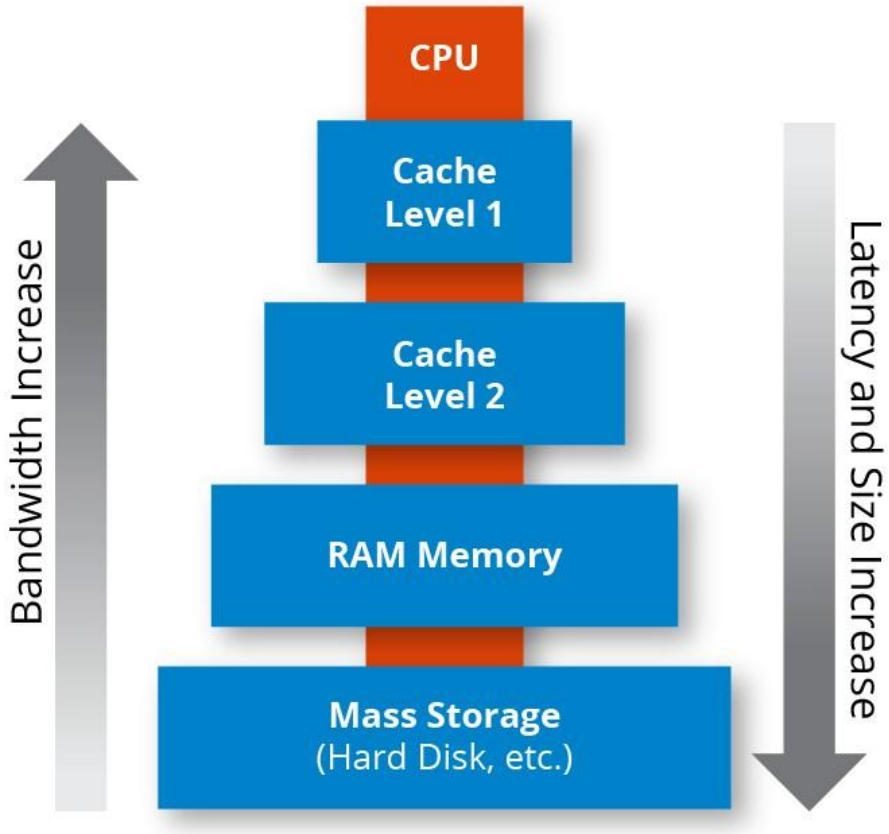
Spark-caching

Example:

- Dataset: 1 million student records
- Task:
  - Count students per major
  - Find average GPA
  - Filter top students

Without cache → data read 3 times
With cache → data read **once**

Caching = keeping data ready in RAM so Spark doesn't go back to disk every time



Spark-caching – PySpark Example

Python

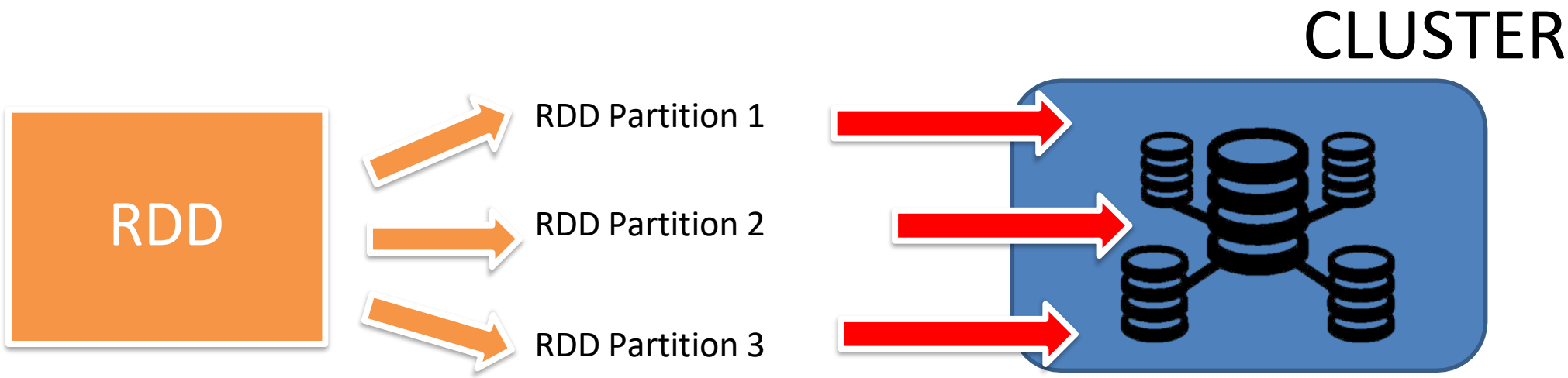
Run

```python
# Load dataset
df = spark.read.csv("students.csv", header=True, inferSchema=True)

# Cache the dataset in memory
df.cache()

# First operation
df.groupBy("major").count().show()

# Second operation
df.selectExpr("avg(gpa)").show()

# Third operation
df.filter(df.gpa > 3.5).show()
```

- Notice we run multiple operations on the same dataset.
- Without cache, Spark would reload the data each time. With cache, it stays in memory.

Spark RDD

- Spark resilient distributed dataset is the spark's core abstraction
- RDD is the immutable distributed collection of objects
- Spark distributes data to different node across the cluster to achieve parallelization



Spark RDD

Think of RDD as:

- A big dataset split into smaller pieces
- Each piece is processed on a different machine

Example:

Dataset = [1, 2, 3, 4, 5, 6]

Split into partitions:

- Machine 1 → [1, 2]
- Machine 2 → [3, 4]
- Machine 3 → [5, 6]

➔ Each machine works **at the same time (parallel processing)**

Spark RDD

Key Properties (simplified):

- **Distributed** → data is split across machines
- **Immutable** → cannot change, only create new RDD
- **Parallel** → operations happen simultaneously

Key Idea:

- **RDD** = Distributed data + Parallel processing

Spark RDD

- To create a RDD we can do by:
  - Loading an external Dataset (from local file system, HDFS, Cassandra etc.)
    - Ex: temperatureRDD = sc.textFile(“/path/to/temperature.txt”)
  - Distributing a collection of objects through **parallelization**
    - Ex: colorsRDD = sc.parallelize(List[“red”,”blue”])
- RDD can be operated on in parallel distData.reduce((a,b)->a+b)

Spark RDD - Parallelization

- An important parameter for parallel collections is the **number of partitions to cut the dataset** into
- Spark will run one task for each partition on the cluster
- By default, Spark sets the number of partitions automatically based on the cluster
- Alternatively, the number of partitions can be set manually by passing it as a **second parameter** to parallelize → sc.parallelize(data,1)

Spark RDD – External Dataset

- Spark supports several data formats of datasets – for Python:
  - SparkContext.wholeTextFiles for small text files
  - RDD.saveAsPickleFile and SparkContext.pickleFile for pickled Python objects
  - SequenceFile (flat, binary file type that serves as a container for data to be used in Apache Hadoop distributed computing projects)
  - Hadoop Input/Output Formats

Spark RDD – External Dataset

- If spark uses a dataset from the local filesystem, then we have to be sure that this dataset or file is accessible from all worker nodes

- All of Spark's file based input methods support compressed files and wildcards
  - Ex: textFile("/my/directory/*.txt")

- The textFile method also takes an optional second argument for controlling the number of partitions of the file – same as within the parallelize method (If we are using HDFS with Spark, then default block size is 128 MB as seen in previous lectures)

Spark RDD Operations

Transformations

Creates a new dataset from an existing one

Lazy computation

Can persist an RDD in memory using persist or cache (Spark will keep the elemts in the cluster for future access)

Actions

Return a value to the program after running a computation

Spark RDD Operations

```
JavaRDD lines = sc.textFile("data.txt");
JavaRDD lineLengths = lines.map(s -> s.length());
int totalLength = lineLengths.reduce((a, b) -> a + b);
```

- Lines and linelengths are not computed, they are pointers to internal structures that remember the operations
- When reduce is run, Spark breaks the computation into tasks to run on separate machines
- To cache lineLengths -> lineLengths.persist(StorageLevel.MEMORY_ONLY());
- We can pass functions to spark by either implementing a function interface (next slide) or by using lambda expression (as per the above example)

Spark RDD Operations

```
class GetLength implements Function<String, Integer> {
   public Integer call(String s) {
     return s.length();
  }
}
class Sum implements Function2<Integer, Integer, Integer> {
    public Integer call(Integer a, Integer b) {
      return a + b;
    }
}
JavaRDD<String> lines = sc.textFile("data.txt");
JavaRDD<Integer> lineLengths = lines.map(new GetLength());
int totalLength = lineLengths.reduce(new Sum());
```

Spark RDD – Transformation functions

map(*func*)	Return a new distributed dataset formed by passing each element of the source through a function *func*.
filter(*func*)	Return a new dataset formed by selecting those elements of the source on which *func* returns true.
flatMap(*func*)	Similar to map, but each input item can be mapped to 0 or more output items (so *func* should return a Seq rather than a single item).
union(*otherDataset*)	Return a new dataset that contains the union of the elements in the source dataset and the argument.
intersection(*otherDataset*)	Return a new RDD that contains the intersection of elements in the source dataset and the argument.

Spark RDD – Actions functions

reduce(*func*)	Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect()	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count()	Return the number of elements in the dataset.
first()	Return the first element of the dataset (similar to take(1)).
take(*n*)	Return an array with the first *n* elements of the dataset.

RDD Operations – Example

Python

Run

```python
# Create RDD
rdd = sc.parallelize([1, 2, 3, 4, 5, 6])

# Transformation (lazy)
rdd2 = rdd.map(lambda x: x * 2)

# Transformation (lazy)
rdd3 = rdd2.filter(lambda x: x > 5)

# Action (triggers computation)
result = rdd3.collect()

print(result)
```

Output:

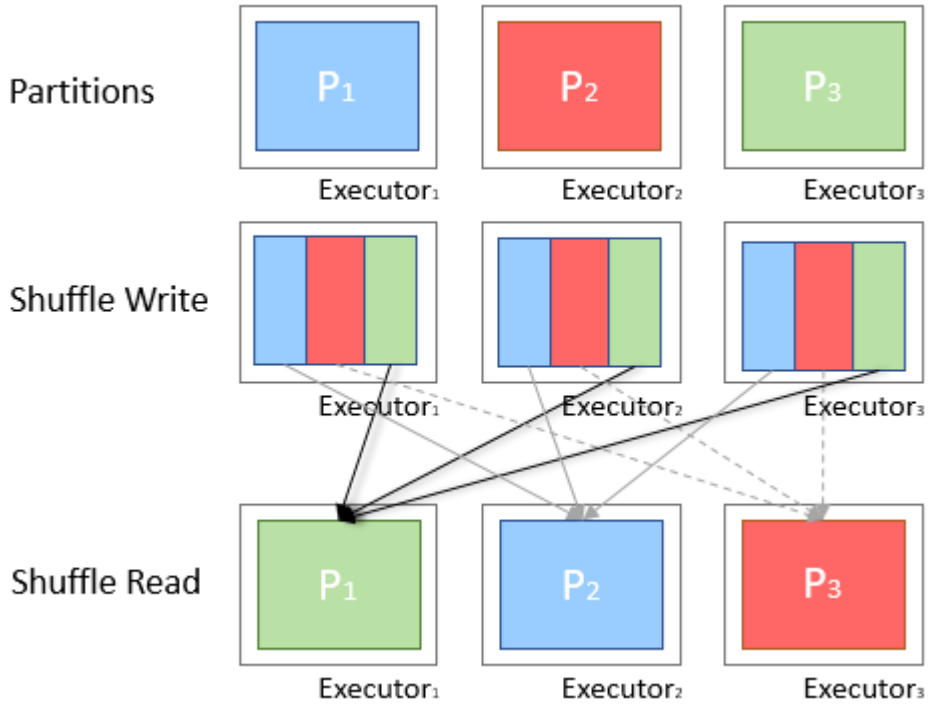
Python

Run

```
[6, 8, 10, 12]
```

Spark shuffle operations

- Certain operations within Spark trigger an event known as the shuffle.
- A **shuffle** occurs when data is rearranged between partitions. This is required when a transformation requires information from other partitions, such as summing all the values in a column. Spark will gather the required data from each partition and combine it into a new partition, likely on a different executor.



Spark shuffle operations

Task:
Count number of students per major

Before Shuffle:
(each partition has mixed data)

- Partition 1 → (CS, Math)
- Partition 2 → (CS, Bio)
- Partition 3 → (Math, Bio)

Shuffle Happens:
Spark reorganizes data so same keys go together

After Shuffle:

- Partition A → (CS, CS)
- Partition B → (Math, Math)
- Partition C → (Bio, Bio)

Then:

- Spark can easily compute counts per group

Key Idea:

- Shuffle = moving data across machines to group related data

Shuffle Example (reduceByKey)

Python

Run

```python
# Create RDD (student major data)
rdd = sc.parallelize([
    ("CS", 1), ("Math", 1), ("CS", 1),
    ("Bio", 1), ("Math", 1), ("Bio", 1)
])

# Shuffle happens here
result = rdd.reduceByKey(lambda a, b: a + b)

# Trigger execution
print(result.collect())
```

Output:

Python

Run

```
[('CS', 2), ('Math', 2), ('Bio', 2)]
```

Spark RDD Persistence

- Each node can store in its memory the partitions and use the value in other actions resulting in much faster operations (cache() and persist())
- Caching is fault tolerant – If any partition of an RDD is lost, it will be automatically be recomputed (because of the RESILIENT characteristics – recomputed through history of activity)
- Using the persist() method we can decide at which storage level we want to store the data:
  - MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY
- Spark automatically drops out old data partitions in a Least-Recently-Used (LRU) fashion.
- To manually drop cached data we can use the unpersist() method

Spark RDD Persistence

Use caching when:

- You reuse the same dataset multiple times
- You run multiple operations on the same RDD
- The computation is expensive

```python
df = spark.read.csv("big_data.csv")

df.cache()

df.count()
df.filter(...).count()
df.groupBy(...).count()
```

Spark Shared Variables – Broadcast variables

- When we run our Spark operations across the cluster, as seen before, tasks will be distributed across the cluster and the data as well.
- This could lead to communication cost problems since we might be transporting lots of data with each task on each node
- This is why Spark introduces Broadcast Variables
- The broadcast variables keep a read-only variable cached on each machine rather than shipping a copy of it with tasks.
- Spark distributes broadcast variables using efficient broadcast algorithms to reduce communication cost
- Broadcast broadcastVar = sc.broadcast(new int[] {1, 2, 3}); broadcastVar.value();

Spark Shared Variables – Broadcast variables

Without Broadcast:

- Small dataset (e.g., lookup table) is sent **with every task**
- Repeated many times across the cluster

➔ High communication cost

With Broadcast:

- Small dataset is sent **once to each machine**
- Stored in memory and reused

➔ Much faster and efficient

Example:

- Large dataset: user transactions
- Small dataset: country codes

Instead of sending country codes every time

Broadcast it once to all nodes

Key Idea:

- Broadcast = send once, reuse everywhere

Spark Shared Variables – Broadcast variables

Python

Run

```python
# Large dataset (RDD)
rdd = sc.parallelize([
    ("Alice", "US"),
    ("Bob", "FR"),
    ("Charlie", "US"),
    ("David", "LB")
])

# Small dataset (lookup list)
valid_countries = ["US", "FR"]

# Broadcast the small dataset
broadcast_countries = sc.broadcast(valid_countries)

# Use broadcast in filtering
filtered_rdd = rdd.filter(lambda x: x[1] in broadcast_countries.value)

print(filtered_rdd.collect())
```

Output:

Python

Run

```
[('Alice', 'US'), ('Bob', 'FR'), ('Charlie', 'US')]
```

Spark Shared Variables – Accumulators

- Outside the scope of transformations and actions, we might need to perform some count/sum operations across the cluster
- Accumulators are simply variables that are meant to count something—hence the name "accumulator." You can specify an accumulator with a default value. If you're starting from scratch, the number would typically be from 0. Accumulators support associative and commutative operations so it can run in parallel
- But why do we need Accumulators ? Can't we just use normal variable and count across the cluster ?
  - The simple answer is that counting values will be **local** to the current executor and updated value is not relayed back to the driver. Accumulators are here to fix this issue.
- Example:
  - Count the number of blank line in a file

```
Kolkata Central Avenue Groceries 233.65

Kolkata Bowbazar Hair Care 198.99
Bad data packet
Kolkata Amherst Street Beverages 0
```

```
var blankLines: Int = 0

sc.textFile("some log file", 4)
    .foreach { line =>
        if (line.length() == 0) blankLines += 1
    }

println(s"Blank Lines=$blankLines")
```

```
val blankLines = sc.accumulator(0, "Blank Lines")

sc.textFile("some log file", 4)
    .foreach { line =>
        if (line.length() == 0) blankLines += 1
    }

println(s"\tBlank Lines=${blankLines.value}")
```

Accumulators – Example with Output

Python

Run

```python
# Create RDD
rdd = sc.parallelize([1, 2, 3, 4, 5])

# Normal variable (won't work properly)
counter = 0

def count_func(x):
    global counter
    if x > 2:
        counter += 1

rdd.foreach(count_func)

print(counter)
```

Output:

Python

Run

```
0
```

Accumulators – Example with Output

Python

```python
# Create accumulator
acc = sc.accumulator(0)

def count_func(x):
    if x > 2:
        acc.add(1)

rdd.foreach(count_func)

print(acc.value)
```

Output:

Python

```
3
```

"In distributed systems, each machine has its own copy of variables. Accumulators are how we safely combine results."

Spark – A self Contained application

- Simply said, a self contained application have no external dependencies while running it in production environments
- A self-contained application consists of a single, installable bundle that contains your application and a copy of the JRE or any Runtime environment needed to run the application. When the application is installed, it behaves the in the same way as any native application.
- We will see next how

Spark – A self Contained application

```
"""SimpleApp.py"""
from pyspark.sql import SparkSession

logFile = "YOUR_SPARK_HOME/README.md"  # Should be some file on your system
spark = SparkSession.builder.appName("SimpleApp").getOrCreate()
logData = spark.read.text(logFile).cache()

numAs = logData.filter(logData.value.contains('a')).count()
numBs = logData.filter(logData.value.contains('b')).count()

print("Lines with a: %i, lines with b: %i" % (numAs, numBs))

spark.stop()
```

Spark – A self Contained application

After running pip install pyspark, we can run our code as per the below

```
# Use the Python interpreter to run your application
$ python SimpleApp.py
...
Lines with a: 46, Lines with b: 23
```

What Did We Just Build?

This is a complete Spark application:

- Reads data
- Processes it using Spark
- Produces results
- Runs independently

Key Components:

- `SparkSession` → entry point
- Data loading → `read()`
- Transformations → `filter()`
- Actions → `count()`
- Output → `print()`

Key Idea:

- A Spark application = data + processing + execution